AWS Cloud Services for Data Science Interview Questions & Answers Cheat Sheet

Table of Contents

AWS Fundamentals

Q1: What are the core AWS services needed for a data science project?

Answer:

Essential AWS Services for Data Science:

Category	Services	Use Case
Compute	EC2, Lambda, Batch	Training models, serverless processing
Storage	S3, EBS, EFS	Data lakes, file systems, databases
Database	RDS, DynamoDB, Redshift	OLTP, NoSQL, data warehousing
Analytics	EMR, Glue, Athena	Big data processing, ETL, queries
ML Services	SageMaker, Comprehend, Rekognition	Model development, NLP, computer vision
Streaming	Kinesis, MSK	Real-time data processing
Security	IAM, KMS, Secrets Manager	Access control, encryption
Monitoring	CloudWatch, CloudTrail	Monitoring, logging, auditing

Typical Data Science Architecture:

```
Data Sources → Kinesis/S3 → Glue (ETL) → Redshift/Athena → SageMaker → API Gateway/La
```

Q2: Explain AWS regions, availability zones, and edge locations.

Answer:

AWS Global Infrastructure:

1. Regions:

- Geographic areas with multiple data centers
- Currently 80+ availability zones across 25+ regions
- Each region is completely independent
- Choose based on: latency, data sovereignty, feature availability, cost

2. Availability Zones (AZs):

- One or more discrete data centers within a region
- Connected via high-bandwidth, low-latency networking
- Physically separated to avoid single points of failure
- Example: `us-east-1a`, `us-east-1b`

3. Edge Locations:

- CDN endpoints for CloudFront
- 400+ edge locations globally
- Cache content closer to users
- Reduce latency for global applications

Data Science Implications:

```
# Multi-AZ deployment for high availability
import boto3

# Deploy across multiple AZs
availability_zones = ['us-east-1a', 'us-east-1b', 'us-east-1c']

# SageMaker training job in specific AZ
sagemaker = boto3.client('sagemaker')
training_job = sagemaker.create_training_job(
    TrainingJobName='ml-training',
    RoleArn='arn:aws:iam::account:role/SageMakerRole',
    AlgorithmSpecification={
        'TrainingImage': 'your-training-image',
        'TrainingInputMode': 'File'
    },
    InputDataConfig=[{
```

```
        'ChannelName': 'training',
        'DataSource': {
            'S3DataSource': {
                'S3DataType': 'S3Prefix',
                'S3Uri': 's3://your-bucket/training-data/',
                'S3DataDistributionType': 'FullyReplicated'
            }
        }
    }],
    ResourceConfig={
        'InstanceType': 'ml.m5.xlarge',
        'InstanceCount': 1,
        'VolumeSizeInGB': 30
    }
)
```

Compute Services (EC2, Lambda)

Q3: When would you use EC2 vs Lambda for data processing?

Answer:

Comparison of EC2 vs Lambda:

Aspect	EC2	Lambda
Use Case	Long-running processes, model training	Event-driven, serverless functions
Duration	Unlimited	15 minutes maximum
Scaling	Manual/Auto Scaling Groups	Automatic, up to 1000 concurrent
Cost	Pay per instance hour	Pay per request and compute time
Cold Start	No cold start	Cold start latency
State	Persistent	Stateless

EC2 Use Cases:

```
# Long-running ML training job
# EC2 Instance for deep learning
instance_config = {
    'ImageId': 'ami-0c7217cdde317cfec',  # Deep Learning AMI
    'InstanceType': 'p3.2xlarge',       # GPU instance
    'KeyName': 'your-key-pair',
    'SecurityGroupIds': ['sg-12345'],
    'UserData': '''
#!/bin/bash
# Install dependencies
pip install torch torchvision
# Download training data from S3
aws s3 sync s3://your-bucket/data /data/
# Start training
```

```
python train_model.py
    '''
}
```

Lambda Use Cases:

```
import json
import boto3
import pandas as pd
from io import StringIO

def lambda_handler(event, context):
    """
    Process new files uploaded to S3
    Triggered by S3 events
    """
    s3 = boto3.client('s3')

    # Get bucket and file info from S3 event
    bucket = event['Records'][0]['s3']['bucket']['name']
    key = event['Records'][0]['s3']['object']['key']

    # Read CSV file from S3
    obj = s3.get_object(Bucket=bucket, Key=key)
    df = pd.read_csv(StringIO(obj['Body'].read().decode('utf-8')))

    # Basic data processing
    df_processed = df.dropna().groupby('category').agg({
        'value': ['mean', 'sum', 'count']
    }).round(2)

    # Save processed data back to S3
    csv_buffer = StringIO()
    df_processed.to_csv(csv_buffer)

    s3.put_object(
        Bucket=bucket,
        Key=f"processed/{key}",
        Body=csv_buffer.getvalue()
    )

    return {
        'statusCode': 200,
        'body': json.dumps(f'Processed {len(df)} records')
    }
```

Q4: How do you optimize EC2 costs for machine learning workloads?

Answer:

Cost Optimization Strategies:

1. Instance Types & Sizing:

```
# Choose right instance types
instance_recommendations = {
    'data_preprocessing': ['m5.large', 'c5.xlarge'],    # CPU-optimized
    'model_training': ['p3.2xlarge', 'p4d.xlarge'],    # GPU instances
    'inference': ['c5.large', 'm5.xlarge'],            # General purpose
    'batch_processing': ['r5.xlarge', 'x1e.xlarge']    # Memory-optimized
}

# Use appropriate storage
storage_config = {
    'training_data': 'gp3',      # General Purpose SSD
    'model_artifacts': 's3',     # Object storage
    'temporary_files': 'io2'     # High performance SSD
}
```

2. Spot Instances:

```
import boto3

def create_spot_training_job():
    sagemaker = boto3.client('sagemaker')

    response = sagemaker.create_training_job(
        TrainingJobName='spot-training-job',
        RoleArn='arn:aws:iam::account:role/SageMakerRole',
        ResourceConfig={
            'InstanceType': 'ml.p3.2xlarge',
            'InstanceCount': 1,
            'VolumeSizeInGB': 100,
            'VolumeKmsKeyId': 'string'
        },
        StoppingCondition={
            'MaxRuntimeInSeconds': 86400
        },
        # Enable Spot instances (up to 90% cost savings)
        EnableManagedSpotTraining=True,
        CheckpointConfig={
            'S3Uri': 's3://your-bucket/checkpoints/',
            'LocalPath': '/opt/ml/checkpoints'
        }
    )

    return response

# Auto Scaling for inference endpoints
def configure_auto_scaling():
    autoscaling = boto3.client('application-autoscaling')

    # Register scalable target
    autoscaling.register_scalable_target(
        ServiceNamespace='sagemaker',
        ResourceId='endpoint/your-endpoint/variant/your-variant',
        ScalableDimension='sagemaker:variant:DesiredInstanceCount',
        MinCapacity=1,
```

```
        MaxCapacity=10
    )

    # Create scaling policy
    autoscaling.put_scaling_policy(
        PolicyName='cpu-scaling-policy',
        ServiceNamespace='sagemaker',
        ResourceId='endpoint/your-endpoint/variant/your-variant',
        ScalableDimension='sagemaker:variant:DesiredInstanceCount',
        PolicyType='TargetTrackingScaling',
        TargetTrackingScalingPolicyConfiguration={
            'TargetValue': 70.0,
            'PredefinedMetricSpecification': {
                'PredefinedMetricType': 'SageMakerVariantInvocationsPerInstance'
            }
        }
    )
```

3. Reserved Instances & Savings Plans:

```
# Cost calculation example
cost_comparison = {
    'on_demand': {
        'p3.2xlarge': 3.06,  # per hour
        'monthly_cost': 3.06 * 24 * 30,
        'use_case': 'Development, testing'
    },
    'reserved_1_year': {
        'p3.2xlarge': 1.84,  # per hour (40% savings)
        'monthly_cost': 1.84 * 24 * 30,
        'use_case': 'Predictable workloads'
    },
    'spot_instance': {
        'p3.2xlarge': 0.92,  # per hour (70% savings)
        'monthly_cost': 0.92 * 24 * 30,
        'use_case': 'Fault-tolerant workloads'
    }
}
```

Storage Services (S3, EBS, EFS)

Q5: How do you design a data lake architecture using S3?

Answer:

S3 Data Lake Architecture:

1. Data Lake Structure:

```
s3://data-lake-bucket/
├── raw/                # Landing zone for raw data
```

```
│   ├── year=2024/
│   ├── month=01/
│   └── day=15/
├── processed/           # Cleaned and transformed data
│   ├── bronze/         # Raw data with basic cleaning
│   ├── silver/         # Business logic applied
│   └── gold/           # Analytics-ready data
├── models/             # ML model artifacts
├── features/           # Feature store
└── logs/              # Application logs
```

2. Storage Classes Optimization:

```
import boto3

def setup_lifecycle_policy():
    s3 = boto3.client('s3')

    lifecycle_config = {
        'Rules': [
            {
                'ID': 'data-lifecycle',
                'Status': 'Enabled',
                'Filter': {'Prefix': 'raw/'},
                'Transitions': [
                    {
                        'Days': 30,
                        'StorageClass': 'STANDARD_IA'  # Infrequent Access
                    },
                    {
                        'Days': 90,
                        'StorageClass': 'GLACIER'      # Long-term archive
                    },
                    {
                        'Days': 365,
                        'StorageClass': 'DEEP_ARCHIVE' # Lowest cost
                    }
                ]
            },
            {
                'ID': 'processed-data',
                'Status': 'Enabled',
                'Filter': {'Prefix': 'processed/'},
                'Transitions': [
                    {
                        'Days': 90,
                        'StorageClass': 'STANDARD_IA'
                    }
                ]
            }
        ]
    }

    s3.put_bucket_lifecycle_configuration(
        Bucket='data-lake-bucket',
```

```
        LifecycleConfiguration=lifecycle_config
    )
```

3. Data Partitioning Strategy:

```
import pandas as pd
from datetime import datetime

def partition_and_upload_data(df, bucket, prefix):
    """
    Partition data by date and upload to S3
    Enables efficient querying with Athena
    """
    s3 = boto3.client('s3')

    # Add partition columns
    df['year'] = df['timestamp'].dt.year
    df['month'] = df['timestamp'].dt.month
    df['day'] = df['timestamp'].dt.day

    # Group by partition columns
    for (year, month, day), group in df.groupby(['year', 'month', 'day']):

        # Create partition path
        partition_path = f"{prefix}/year={year}/month={month:02d}/day={day:02d}/"

        # Save as parquet for better performance
        parquet_buffer = BytesIO()
        group.drop(['year', 'month', 'day'], axis=1).to_parquet(
            parquet_buffer,
            engine='pyarrow',
            compression='snappy'
        )

        # Upload to S3
        s3.put_object(
            Bucket=bucket,
            Key=f"{partition_path}data_{datetime.now().strftime('%H%M%S')}.parquet",
            Body=parquet_buffer.getvalue()
        )

# Usage
data = pd.read_csv('large_dataset.csv')
data['timestamp'] = pd.to_datetime(data['timestamp'])
partition_and_upload_data(data, 'data-lake-bucket', 'raw/sales-data')
```

Q6: What are the different S3 storage classes and when to use each?

Answer:

S3 Storage Classes Comparison:

Storage Class	Durability	Availability	Use Case	Cost
Standard	99.999999999%	99.99%	Frequently accessed data	Highest
Standard-IA	99.999999999%	99.9%	Infrequently accessed	Medium
One Zone-IA	99.999999999%	99.5%	Non-critical, infrequent	Lower
Glacier Instant	99.999999999%	99.9%	Archive with instant retrieval	Low
Glacier Flexible	99.999999999%	99.99%	Long-term archive	Very Low
Deep Archive	99.999999999%	99.99%	Long-term, rarely accessed	Lowest

Implementation Example:

```
import boto3
from botocore.exceptions import ClientError

class S3DataManager:
    def __init__(self):
        self.s3 = boto3.client('s3')

    def upload_training_data(self, file_path, bucket, key):
        """Upload frequently accessed training data to Standard"""
        self.s3.upload_file(
            file_path, bucket, key,
            ExtraArgs={
                'StorageClass': 'STANDARD',
                'Metadata': {
                    'data-type': 'training',
                    'access-frequency': 'high'
                }
            }
        )

    def archive_old_models(self, bucket, prefix):
        """Move old model artifacts to Glacier"""
        paginator = self.s3.get_paginator('list_objects_v2')

        for page in paginator.paginate(Bucket=bucket, Prefix=prefix):
            if 'Contents' in page:
                for obj in page['Contents']:
                    # Check if object is older than 30 days
                    age = (datetime.now(timezone.utc) - obj['LastModified']).days

                    if age &gt; 30:
                        # Copy to Glacier
                        self.s3.copy_object(
                            CopySource={'Bucket': bucket, 'Key': obj['Key']},
                            Bucket=bucket,
                            Key=obj['Key'],
                            StorageClass='GLACIER'
                        )

    def restore_archived_data(self, bucket, key, days=7):
        """Restore data from Glacier for analysis"""
```

```
        try:
            self.s3.restore_object(
                Bucket=bucket,
                Key=key,
                RestoreRequest={
                    'Days': days,
                    'GlacierJobParameters': {
                        'Tier': 'Standard'  # Standard, Expedited, or Bulk
                    }
                }
            )
            return "Restore initiated"
        except ClientError as e:
            return f"Error: {e}"

    def intelligent_tiering_setup(self, bucket):
        """Setup Intelligent Tiering for automatic optimization"""
        configuration = {
            'Id': 'intelligent-tiering-config',
            'Status': 'Enabled',
            'Filter': {'Prefix': 'data/'},
            'Tiering': {
                'Days': 1,
                'StorageClass': 'INTELLIGENT_TIERING'
            }
        }

        self.s3.put_bucket_intelligent_tiering_configuration(
            Bucket=bucket,
            Id='intelligent-tiering-config',
            IntelligentTieringConfiguration=configuration
        )
```

Database Services (RDS, DynamoDB, Redshift)

Q7: When should you use RDS vs DynamoDB vs Redshift?

Answer:

Database Service Comparison:

Service	Type	Use Case	Scaling	Query Language
RDS	Relational (OLTP)	Transactional apps, structured data	Vertical	SQL
DynamoDB	NoSQL	High-traffic apps, IoT, gaming	Horizontal	DynamoDB API
Redshift	Data Warehouse (OLAP)	Analytics, BI, reporting	Horizontal	SQL

RDS Use Cases:

```
import boto3
import pandas as pd
from sqlalchemy import create_engine

# RDS for structured transaction data
def setup_rds_connection():
    # RDS connection for user data, transactions
    engine = create_engine(
        'postgresql://username:password@rds-endpoint:5432/database'
    )

    # Store model metadata
    model_metadata = pd.DataFrame({
        'model_id': ['model_001', 'model_002'],
        'model_name': ['fraud_detection', 'recommendation'],
        'accuracy': [0.95, 0.87],
        'created_at': pd.Timestamp.now()
    })

    model_metadata.to_sql('model_registry', engine, if_exists='append')

    return engine

# Multi-AZ deployment for high availability
rds_config = {
    'DBInstanceIdentifier': 'prod-ml-database',
    'DBInstanceClass': 'db.r5.xlarge',
    'Engine': 'postgres',
    'MasterUsername': 'mluser',
    'AllocatedStorage': 500,
    'MultiAZ': True,              # High availability
    'BackupRetentionPeriod': 7,   # 7-day backup
    'StorageEncrypted': True,     # Encryption at rest
    'VpcSecurityGroupIds': ['sg-12345']
}
```

DynamoDB Use Cases:

```
import boto3
from decimal import Decimal
from datetime import datetime

class DynamoMLStore:
    def __init__(self):
        self.dynamodb = boto3.resource('dynamodb')
        self.predictions_table = self.dynamodb.Table('ml-predictions')
        self.features_table = self.dynamodb.Table('ml-features')

    def store_prediction(self, user_id, model_id, prediction, confidence):
        """Store real-time ML predictions"""
        self.predictions_table.put_item(
            Item={
                'user_id': user_id,
                'timestamp': int(datetime.now().timestamp()),
```

```
                'model_id': model_id,
                'prediction': Decimal(str(prediction)),
                'confidence': Decimal(str(confidence)),
                'ttl': int(datetime.now().timestamp()) + 86400  # 24 hours
            }
        )

    def get_user_features(self, user_id):
        """Retrieve user features for ML inference"""
        response = self.features_table.get_item(
            Key={'user_id': user_id}
        )
        return response.get('Item', {})

    def batch_write_features(self, features_batch):
        """Batch write feature vectors"""
        with self.features_table.batch_writer() as batch:
            for features in features_batch:
                batch.put_item(Item=features)

# DynamoDB auto-scaling configuration
def setup_dynamodb_autoscaling():
    autoscaling = boto3.client('application-autoscaling')

    # Register scalable target
    autoscaling.register_scalable_target(
        ServiceNamespace='dynamodb',
        ResourceId='table/ml-predictions',
        ScalableDimension='dynamodb:table:ReadCapacityUnits',
        MinCapacity=5,
        MaxCapacity=1000
    )
```

Redshift Use Cases:

```
import pandas as pd
import psycopg2
from sqlalchemy import create_engine

def setup_redshift_analytics():
    # Redshift for large-scale analytics
    engine = create_engine(
        'redshift+psycopg2://user:password@redshift-cluster:5439/analytics'
    )

    # Complex analytical queries
    feature_engineering_query = """
    SELECT
        user_id,
        AVG(purchase_amount) as avg_purchase,
        COUNT(*) as total_transactions,
        MAX(purchase_date) as last_purchase,
        DATE_DIFF('day', MIN(purchase_date), MAX(purchase_date)) as customer_lifetime_day
        SUM(CASE WHEN category = 'electronics' THEN 1 ELSE 0 END) as electronics_purchase
    FROM transactions
```

```
    WHERE purchase_date &gt;= DATEADD(month, -12, CURRENT_DATE)
    GROUP BY user_id
    HAVING COUNT(*) &gt; 5
    """

    features_df = pd.read_sql(feature_engineering_query, engine)

    # Window functions for advanced analytics
    cohort_analysis = """
    WITH user_cohorts AS (
        SELECT
            user_id,
            DATE_TRUNC('month', MIN(purchase_date)) as cohort_month
        FROM transactions
        GROUP BY user_id
    ),
    cohort_data AS (
        SELECT
            c.cohort_month,
            DATE_TRUNC('month', t.purchase_date) as period_month,
            COUNT(DISTINCT t.user_id) as active_users
        FROM user_cohorts c
        JOIN transactions t ON c.user_id = t.user_id
        GROUP BY 1, 2
    )
    SELECT
        cohort_month,
        period_month,
        active_users,
        LAG(active_users) OVER (PARTITION BY cohort_month ORDER BY period_month) as prev_
    FROM cohort_data
    ORDER BY cohort_month, period_month
    """

    return pd.read_sql(cohort_analysis, engine)
```

Q8: How do you implement a feature store using DynamoDB?

Answer:

Feature Store Architecture:

```
import boto3
import json
from typing import Dict, List, Any
from decimal import Decimal
from datetime import datetime, timedelta

class DynamoDBFeatureStore:
    def __init__(self, table_name: str):
        self.dynamodb = boto3.resource('dynamodb')
        self.table = self.dynamodb.Table(table_name)

    def create_feature_store_table(self):
```

```
        """Create optimized DynamoDB table for feature store"""
        try:
            table = self.dynamodb.create_table(
                TableName='ml-feature-store',
                KeySchema=[
                    {
                        'AttributeName': 'entity_id',    # Partition key
                        'KeyType': 'HASH'
                    },
                    {
                        'AttributeName': 'feature_group',  # Sort key
                        'KeyType': 'RANGE'
                    }
                ],
                AttributeDefinitions=[
                    {
                        'AttributeName': 'entity_id',
                        'AttributeType': 'S'
                    },
                    {
                        'AttributeName': 'feature_group',
                        'AttributeType': 'S'
                    },
                    {
                        'AttributeName': 'event_time',
                        'AttributeType': 'N'
                    }
                ],
                GlobalSecondaryIndexes=[
                    {
                        'IndexName': 'FeatureGroupIndex',
                        'KeySchema': [
                            {
                                'AttributeName': 'feature_group',
                                'KeyType': 'HASH'
                            },
                            {
                                'AttributeName': 'event_time',
                                'KeyType': 'RANGE'
                            }
                        ],
                        'Projection': {'ProjectionType': 'ALL'},
                        'ProvisionedThroughput': {
                            'ReadCapacityUnits': 10,
                            'WriteCapacityUnits': 10
                        }
                    }
                ],
                BillingMode='PAY_PER_REQUEST'
            )

            table.wait_until_exists()
            return table

        except Exception as e:
            print(f"Error creating table: {e}")
```

```
            return None

    def put_features(self, entity_id: str, feature_group: str, features: Dict[str, Any]):
        """Store feature vector with versioning"""

        # Convert float to Decimal for DynamoDB
        def convert_floats(obj):
            if isinstance(obj, float):
                return Decimal(str(obj))
            elif isinstance(obj, dict):
                return {k: convert_floats(v) for k, v in obj.items()}
            elif isinstance(obj, list):
                return [convert_floats(item) for item in obj]
            return obj

        item = {
            'entity_id': entity_id,
            'feature_group': feature_group,
            'features': convert_floats(features),
            'event_time': int(datetime.now().timestamp()),
            'ingestion_time': datetime.now().isoformat(),
            'ttl': int((datetime.now() + timedelta(days=30)).timestamp())
        }

        self.table.put_item(Item=item)

    def get_features(self, entity_id: str, feature_group: str = None) -&gt; Dict:
        """Retrieve latest features for an entity"""

        if feature_group:
            response = self.table.get_item(
                Key={
                    'entity_id': entity_id,
                    'feature_group': feature_group
                }
            )
            return response.get('Item', {})
        else:
            # Get all feature groups for entity
            response = self.table.query(
                KeyConditionExpression='entity_id = :id',
                ExpressionAttributeValues={':id': entity_id}
            )
            return response['Items']

    def get_batch_features(self, entity_ids: List[str], feature_groups: List[str]) -&gt;
        """Batch retrieve features for multiple entities"""

        # Prepare batch get request
        request_items = {}
        keys = []

        for entity_id in entity_ids:
            for feature_group in feature_groups:
                keys.append({
                    'entity_id': entity_id,
```

```
                    'feature_group': feature_group
                })

            request_items[self.table.table_name] = {
                'Keys': keys
            }

            # Execute batch get
            response = self.dynamodb.batch_get_item(RequestItems=request_items)

            # Organize results by entity_id
            results = {}
            for item in response['Responses'][self.table.table_name]:
                entity_id = item['entity_id']
                if entity_id not in results:
                    results[entity_id] = {}
                results[entity_id][item['feature_group']] = item['features']

            return results

        def get_feature_statistics(self, feature_group: str, days: int = 7) -&gt; Dict:
            """Get feature statistics for monitoring"""

            cutoff_time = int((datetime.now() - timedelta(days=days)).timestamp())

            response = self.table.query(
                IndexName='FeatureGroupIndex',
                KeyConditionExpression='feature_group = :fg AND event_time &gt; :time',
                ExpressionAttributeValues={
                    ':fg': feature_group,
                    ':time': cutoff_time
                }
            )

            # Calculate basic statistics
            items = response['Items']
            stats = {
                'total_records': len(items),
                'unique_entities': len(set(item['entity_id'] for item in items)),
                'latest_update': max(item['event_time'] for item in items) if items else 0
            }

            return stats

    # Usage Example
    def implement_feature_store():
        feature_store = DynamoDBFeatureStore('ml-feature-store')

        # Store user features
        user_features = {
            'age': 25.0,
            'income': 50000.0,
            'credit_score': 750.0,
            'account_balance': 12500.50,
            'transaction_frequency': 15.0
        }
```

```
    feature_store.put_features(
        entity_id='user_12345',
        feature_group='demographic_features',
        features=user_features
    )

    # Store transaction features
    transaction_features = {
        'avg_transaction_amount': 125.75,
        'total_transactions_30d': 28.0,
        'unique_merchants_30d': 12.0,
        'weekend_transaction_ratio': 0.35
    }

    feature_store.put_features(
        entity_id='user_12345',
        feature_group='transaction_features',
        features=transaction_features
    )

    # Retrieve features for inference
    features = feature_store.get_batch_features(
        entity_ids=['user_12345', 'user_67890'],
        feature_groups=['demographic_features', 'transaction_features']
    )

    return features
```

Data Analytics & ETL (EMR, Glue, Athena)

Q9: How do you design an ETL pipeline using AWS Glue?

Answer:

AWS Glue ETL Pipeline Architecture:

1. Glue Components:

```
import boto3
import json
from datetime import datetime

class GlueETLPipeline:
    def __init__(self):
        self.glue = boto3.client('glue')

    def create_data_catalog(self):
        """Create Glue Data Catalog for data discovery"""

        # Create database
        database_input = {
            'Name': 'ml_data_catalog',
```

```
            'Description': 'Data catalog for ML pipeline'
        }

        self.glue.create_database(DatabaseInput=database_input)

        # Create table for raw data
        table_input = {
            'Name': 'raw_transactions',
            'StorageDescriptor': {
                'Columns': [
                    {'Name': 'transaction_id', 'Type': 'string'},
                    {'Name': 'user_id', 'Type': 'string'},
                    {'Name': 'amount', 'Type': 'double'},
                    {'Name': 'category', 'Type': 'string'},
                    {'Name': 'timestamp', 'Type': 'timestamp'}
                ],
                'Location': 's3://data-lake/raw/transactions/',
                'InputFormat': 'org.apache.hadoop.mapred.TextInputFormat',
                'OutputFormat': 'org.apache.hadoop.hive.ql.io.HiveIgnoreKeyTextOutputForm
                'SerdeInfo': {
                    'SerializationLibrary': 'org.apache.hadoop.hive.serde2.lazy.LazySimpl
                }
            },
            'PartitionKeys': [
                {'Name': 'year', 'Type': 'string'},
                {'Name': 'month', 'Type': 'string'},
                {'Name': 'day', 'Type': 'string'}
            ]
        }

        self.glue.create_table(
            DatabaseName='ml_data_catalog',
            TableInput=table_input
        )

    def create_etl_job(self):
        """Create Glue ETL job for data transformation"""

        # ETL job script
        etl_script = '''
import sys
from awsglue.transforms import *
from awsglue.utils import getResolvedOptions
from pyspark.context import SparkContext
from awsglue.context import GlueContext
from awsglue.job import Job
from pyspark.sql import functions as F
from pyspark.sql.types import *

# Initialize Glue context
sc = SparkContext()
glueContext = GlueContext(sc)
spark = glueContext.spark_session
job = Job(glueContext)

args = getResolvedOptions(sys.argv, ['JOB_NAME', 'source_database', 'source_table', 'targ
```

```
job.init(args['JOB_NAME'], args)

# Read data from Glue catalog
datasource = glueContext.create_dynamic_frame.from_catalog(
    database=args['source_database'],
    table_name=args['source_table']
)

# Convert to Spark DataFrame for complex transformations
df = datasource.toDF()

# Data cleaning and feature engineering
df_cleaned = df.filter(
    (F.col("amount") &gt; 0) &amp;
    (F.col("amount") &lt; 10000) &amp;
    (F.col("user_id").isNotNull())
)

# Feature engineering
df_features = df_cleaned.withColumn(
    "is_weekend",
    F.dayofweek(F.col("timestamp")).isin([1, 7])
).withColumn(
    "hour_of_day",
    F.hour(F.col("timestamp"))
).withColumn(
    "transaction_category_encoded",
    F.when(F.col("category") == "groceries", 1)
     .when(F.col("category") == "entertainment", 2)
     .when(F.col("category") == "gas", 3)
     .otherwise(0)
).withColumn(
    "amount_log",
    F.log(F.col("amount"))
)

# Aggregations for user-level features
user_features = df_features.groupBy("user_id").agg(
    F.avg("amount").alias("avg_transaction_amount"),
    F.count("transaction_id").alias("total_transactions"),
    F.countDistinct("category").alias("unique_categories"),
    F.stddev("amount").alias("amount_stddev"),
    F.max("timestamp").alias("last_transaction_date")
)

# Convert back to DynamicFrame
output_frame = DynamicFrame.fromDF(user_features, glueContext, "user_features")

# Write to S3 in Parquet format
glueContext.write_dynamic_frame.from_options(
    frame=output_frame,
    connection_type="s3",
    connection_options={
        "path": args['target_path'],
        "partitionKeys": []
    },
```

```
    format="parquet"
)

job.commit()
        '''

        # Save script to S3
        s3 = boto3.client('s3')
        s3.put_object(
            Bucket='glue-scripts-bucket',
            Key='etl-job.py',
            Body=etl_script
        )

        # Create Glue job
        job_response = self.glue.create_job(
            Name='ml-data-preprocessing',
            Role='arn:aws:iam::account:role/GlueServiceRole',
            Command={
                'Name': 'glueetl',
                'ScriptLocation': 's3://glue-scripts-bucket/etl-job.py',
                'PythonVersion': '3'
            },
            DefaultArguments={
                '--enable-metrics': '',
                '--job-language': 'python',
                '--job-bookmark-option': 'job-bookmark-enable',
                '--source_database': 'ml_data_catalog',
                '--source_table': 'raw_transactions',
                '--target_path': 's3://processed-data/user-features/'
            },
            MaxRetries=1,
            Timeout=60,
            GlueVersion='3.0',
            WorkerType='G.1X',
            NumberOfWorkers=2
        )

        return job_response

    def create_workflow(self):
        """Create Glue workflow for job orchestration"""

        workflow_response = self.glue.create_workflow(
            Name='ml-etl-workflow',
            Description='End-to-end ML data pipeline'
        )

        # Create crawler trigger
        crawler_trigger = self.glue.create_trigger(
            Name='start-crawler-trigger',
            Type='ON_DEMAND',
            WorkflowName='ml-etl-workflow',
            Actions=[
                {
                    'CrawlerName': 'raw-data-crawler'
```

```
                }
            ]
        )

        # Create ETL job trigger (depends on crawler)
        etl_trigger = self.glue.create_trigger(
            Name='etl-job-trigger',
            Type='CONDITIONAL',
            WorkflowName='ml-etl-workflow',
            Predicate={
                'Conditions': [
                    {
                        'LogicalOperator': 'EQUALS',
                        'CrawlerName': 'raw-data-crawler',
                        'CrawlState': 'SUCCEEDED'
                    }
                ]
            },
            Actions=[
                {
                    'JobName': 'ml-data-preprocessing'
                }
            ]
        )

        return workflow_response
```

Q10: How do you use Amazon Athena for interactive queries on data lakes?

Answer:

Athena Query Optimization:

1. Table Creation and Partitioning:

```
-- Create external table in Athena
CREATE EXTERNAL TABLE IF NOT EXISTS ml_data.user_transactions (
    transaction_id string,
    user_id string,
    amount double,
    category string,
    merchant string,
    timestamp timestamp
)
PARTITIONED BY (
    year string,
    month string,
    day string
)
STORED AS PARQUET
LOCATION 's3://data-lake/processed/transactions/'
TBLPROPERTIES (
    'projection.enabled' = 'true',
    'projection.year.type' = 'integer',
```

```
    'projection.year.range' = '2020,2025',
    'projection.year.interval' = '1',
    'projection.month.type' = 'integer',
    'projection.month.range' = '01,12',
    'projection.month.interval' = '1',
    'projection.day.type' = 'integer',
    'projection.day.range' = '01,31',
    'projection.day.interval' = '1',
    'storage.location.template' = 's3://data-lake/processed/transactions/year=${year}/mon
);
```

2. Feature Engineering with Athena:

```
import boto3
import pandas as pd

class AthenaFeatureEngineering:
    def __init__(self, database='ml_data', s3_output='s3://athena-results/'):
        self.athena = boto3.client('athena')
        self.database = database
        self.s3_output = s3_output

    def execute_query(self, query):
        """Execute Athena query and wait for completion"""

        response = self.athena.start_query_execution(
            QueryString=query,
            QueryExecutionContext={'Database': self.database},
            ResultConfiguration={'OutputLocation': self.s3_output}
        )

        query_execution_id = response['QueryExecutionId']

        # Wait for query completion
        while True:
            response = self.athena.get_query_execution(
                QueryExecutionId=query_execution_id
            )
            status = response['QueryExecution']['Status']['State']

            if status in ['SUCCEEDED', 'FAILED', 'CANCELLED']:
                break

            time.sleep(1)

        return query_execution_id, status

    def create_user_features(self):
        """Create comprehensive user feature set"""

        feature_query = """
        WITH user_transaction_stats AS (
            SELECT
                user_id,
                COUNT(*) as total_transactions,
```

```
            AVG(amount) as avg_amount,
            STDDEV(amount) as stddev_amount,
            MIN(amount) as min_amount,
            MAX(amount) as max_amount,
            SUM(amount) as total_spent,
            COUNT(DISTINCT category) as unique_categories,
            COUNT(DISTINCT merchant) as unique_merchants,

            -- Time-based features
            COUNT(CASE WHEN EXTRACT(DOW FROM timestamp) IN (0,6) THEN 1 END) as weeke
            COUNT(CASE WHEN EXTRACT(HOUR FROM timestamp) BETWEEN 9 AND 17 THEN 1 END)

            -- Behavioral patterns
            AVG(CASE WHEN category = 'groceries' THEN amount END) as avg_grocery_spen
            COUNT(CASE WHEN category = 'groceries' THEN 1 END) as grocery_transactior

            -- Recency, Frequency, Monetary features
            DATE_DIFF('day', MAX(timestamp), CURRENT_DATE) as days_since_last_transac
            DATE_DIFF('day', MIN(timestamp), MAX(timestamp)) as customer_lifetime_day

        FROM user_transactions
        WHERE year &gt;= '2023'
        GROUP BY user_id
        HAVING COUNT(*) &gt;= 5  -- Minimum transaction threshold
    ),

    user_category_preferences AS (
        SELECT
            user_id,
            category,
            COUNT(*) as category_count,
            SUM(amount) as category_spending,
            ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY COUNT(*) DESC) as catego
        FROM user_transactions
        WHERE year &gt;= '2023'
        GROUP BY user_id, category
    ),

    top_categories AS (
        SELECT
            user_id,
            MAX(CASE WHEN category_rank = 1 THEN category END) as top_category,
            MAX(CASE WHEN category_rank = 2 THEN category END) as second_category,
            MAX(CASE WHEN category_rank = 1 THEN category_spending END) as top_catego
        FROM user_category_preferences
        WHERE category_rank &lt;= 2
        GROUP BY user_id
    )

    SELECT
        uts.*,
        tc.top_category,
        tc.second_category,
        tc.top_category_spending,

        -- Derived features
```

```
            CASE
                WHEN uts.stddev_amount / uts.avg_amount &gt; 0.5 THEN 'high_variability'
                WHEN uts.stddev_amount / uts.avg_amount &gt; 0.2 THEN 'medium_variability
                ELSE 'low_variability'
            END as spending_variability,

            CASE
                WHEN uts.days_since_last_transaction &lt;= 7 THEN 'active'
                WHEN uts.days_since_last_transaction &lt;= 30 THEN 'recent'
                ELSE 'inactive'
            END as user_status,

            -- Normalized features
            (uts.avg_amount - AVG(uts.avg_amount) OVER()) / STDDEV(uts.avg_amount) OVER()
            (uts.total_transactions - AVG(uts.total_transactions) OVER()) / STDDEV(uts.to

        FROM user_transaction_stats uts
        LEFT JOIN top_categories tc ON uts.user_id = tc.user_id
        ORDER BY uts.total_spent DESC
        """

        query_id, status = self.execute_query(feature_query)

        if status == 'SUCCEEDED':
            return f"Features created successfully. Query ID: {query_id}"
        else:
            return f"Query failed. Status: {status}"

    def create_ctas_table(self, table_name, query):
        """Create Table As Select for storing results"""

        ctas_query = f"""
        CREATE TABLE {table_name}
        WITH (
            format = 'PARQUET',
            parquet_compression = 'SNAPPY',
            external_location = 's3://processed-data/{table_name}/',
            partitioned_by = ARRAY['user_status']
        ) AS
        {query}
        """

        return self.execute_query(ctas_query)

# Usage example
def run_athena_feature_engineering():
    athena_fe = AthenaFeatureEngineering()

    # Create user features
    result = athena_fe.create_user_features()
    print(result)

    # Query to validate features
    validation_query = """
    SELECT
        COUNT(*) as total_users,
```

```
        AVG(avg_amount) as overall_avg_amount,
        COUNT(DISTINCT top_category) as unique_top_categories,
        user_status,
        COUNT(*) as users_by_status
    FROM user_features
    GROUP BY user_status
    ORDER BY users_by_status DESC
    """

    validation_id, status = athena_fe.execute_query(validation_query)
    return validation_id, status
```

Machine Learning Services (SageMaker, Comprehend)

Q11: How do you build an end-to-end ML pipeline with SageMaker?

Answer:

Complete SageMaker ML Pipeline:

1. Data Preparation and Feature Engineering:

```
import boto3
import pandas as pd
import sagemaker
from sagemaker import get_execution_role
from sagemaker.sklearn.processing import SKLearnProcessor
from sagemaker.processing import ProcessingInput, ProcessingOutput

class SageMakerMLPipeline:
    def __init__(self):
        self.sagemaker_session = sagemaker.Session()
        self.role = get_execution_role()
        self.bucket = self.sagemaker_session.default_bucket()
        self.region = boto3.Session().region_name

    def create_preprocessing_job(self):
        """Create SageMaker Processing job for data preparation"""

        # Processing script
        preprocessing_script = '''
import argparse
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import train_test_split
import joblib
import os

def preprocess_data():
    parser = argparse.ArgumentParser()
    parser.add_argument('--input-data', type=str, default='/opt/ml/processing/input')
    parser.add_argument('--output-data', type=str, default='/opt/ml/processing/output')
```

```
    args = parser.parse_args()

    # Read raw data
    df = pd.read_csv(f"{args.input_data}/raw_data.csv")

    # Data cleaning
    df = df.dropna()
    df = df[df['amount'] &gt; 0]

    # Feature engineering
    df['hour'] = pd.to_datetime(df['timestamp']).dt.hour
    df['day_of_week'] = pd.to_datetime(df['timestamp']).dt.dayofweek
    df['is_weekend'] = df['day_of_week'].isin([5, 6]).astype(int)

    # Log transformation
    df['amount_log'] = np.log1p(df['amount'])

    # Categorical encoding
    le = LabelEncoder()
    df['category_encoded'] = le.fit_transform(df['category'])

    # Feature scaling
    numeric_features = ['amount', 'amount_log', 'hour']
    scaler = StandardScaler()
    df[numeric_features] = scaler.fit_transform(df[numeric_features])

    # Train/test split
    X = df.drop(['target'], axis=1)
    y = df['target']

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )

    # Save processed data
    pd.concat([X_train, y_train], axis=1).to_csv(
        f"{args.output_data}/train.csv", index=False
    )
    pd.concat([X_test, y_test], axis=1).to_csv(
        f"{args.output_data}/test.csv", index=False
    )

    # Save preprocessors
    joblib.dump(scaler, f"{args.output_data}/scaler.joblib")
    joblib.dump(le, f"{args.output_data}/label_encoder.joblib")

    print("Preprocessing completed successfully")

if __name__ == "__main__":
    preprocess_data()
        '''

        # Save script to S3
        with open('preprocessing.py', 'w') as f:
            f.write(preprocessing_script)
```

```
        # Upload to S3
        s3_preprocessing_code = self.sagemaker_session.upload_data(
            'preprocessing.py',
            bucket=self.bucket,
            key_prefix='code'
        )

        # Create processor
        sklearn_processor = SKLearnProcessor(
            framework_version='0.23-1',
            instance_type='ml.m5.xlarge',
            instance_count=1,
            role=self.role,
            base_job_name='ml-preprocessing'
        )

        # Run processing job
        sklearn_processor.run(
            code='preprocessing.py',
            source_dir=None,
            inputs=[
                ProcessingInput(
                    source='s3://your-bucket/raw-data/',
                    destination='/opt/ml/processing/input',
                    input_name='raw-data'
                )
            ],
            outputs=[
                ProcessingOutput(
                    source='/opt/ml/processing/output',
                    destination=f's3://{self.bucket}/processed-data/',
                    output_name='processed-data'
                )
            ],
            arguments=[]
        )

        return sklearn_processor

    def create_training_job(self, algorithm='xgboost'):
        """Create SageMaker training job"""

        if algorithm == 'xgboost':
            # XGBoost training
            from sagemaker.xgboost.estimator import XGBoost

            xgb_estimator = XGBoost(
                entry_point='train.py',
                source_dir='code',
                framework_version='1.3-1',
                py_version='py3',
                instance_type='ml.m5.xlarge',
                instance_count=1,
                role=self.role,
                hyperparameters={
```

```
                    'objective': 'binary:logistic',
                    'eval_metric': 'auc',
                    'num_round': 100,
                    'max_depth': 6,
                    'eta': 0.1,
                    'subsample': 0.8,
                    'colsample_bytree': 0.8
                },
                use_spot_instances=True,
                max_wait=3600,
                max_run=3600
            )

        elif algorithm == 'sklearn':
            # Custom scikit-learn algorithm
            from sagemaker.sklearn.estimator import SKLearn

            sklearn_estimator = SKLearn(
                entry_point='train.py',
                source_dir='code',
                framework_version='0.23-1',
                py_version='py3',
                instance_type='ml.m5.xlarge',
                role=self.role
            )

            return sklearn_estimator

        # Training data
        train_input = sagemaker.inputs.TrainingInput(
            s3_data=f's3://{self.bucket}/processed-data/train.csv',
            content_type='text/csv'
        )

        validation_input = sagemaker.inputs.TrainingInput(
            s3_data=f's3://{self.bucket}/processed-data/test.csv',
            content_type='text/csv'
        )

        # Start training
        xgb_estimator.fit({
            'train': train_input,
            'validation': validation_input
        })

        return xgb_estimator

    def setup_model_registry(self, estimator, model_name):
        """Register model in SageMaker Model Registry"""

        from sagemaker.model import Model

        model = Model(
            image_uri=estimator.training_image_uri(),
            model_data=estimator.model_data,
            role=self.role,
```

```
            predictor_cls=estimator.__class__.default_predictor_cls
        )

        # Create model package
        model_package = model.register(
            content_types=["text/csv"],
            response_types=["text/csv"],
            inference_instances=["ml.t2.medium", "ml.m5.large"],
            transform_instances=["ml.m5.large"],
            model_package_group_name=model_name,
            approval_status="Approved",
            description="Fraud detection model"
        )

        return model_package

    def create_endpoint(self, model_package):
        """Deploy model to SageMaker endpoint"""

        from sagemaker.model import Model

        model = Model(
            image_uri=model_package.describe()['InferenceSpecification']['Containers'][0]
            model_data=model_package.describe()['InferenceSpecification']['Containers'][0
            role=self.role
        )

        # Deploy with auto-scaling
        predictor = model.deploy(
            initial_instance_count=1,
            instance_type='ml.m5.large',
            endpoint_name='fraud-detection-endpoint'
        )

        # Setup auto-scaling
        client = boto3.client('application-autoscaling')

        client.register_scalable_target(
            ServiceNamespace='sagemaker',
            ResourceId='endpoint/fraud-detection-endpoint/variant/AllTraffic',
            ScalableDimension='sagemaker:variant:DesiredInstanceCount',
            MinCapacity=1,
            MaxCapacity=10
        )

        client.put_scaling_policy(
            PolicyName='sagemaker-scaling-policy',
            ServiceNamespace='sagemaker',
            ResourceId='endpoint/fraud-detection-endpoint/variant/AllTraffic',
            ScalableDimension='sagemaker:variant:DesiredInstanceCount',
            PolicyType='TargetTrackingScaling',
            TargetTrackingScalingPolicyConfiguration={
                'TargetValue': 70.0,
                'PredefinedMetricSpecification': {
                    'PredefinedMetricType': 'SageMakerVariantInvocationsPerInstance'
                }
```

```
            }
        )

        return predictor
```

Q12: How do you implement A/B testing for ML models in production?

Answer:

SageMaker A/B Testing Implementation:

```
import boto3
import json
from datetime import datetime, timedelta

class SageMakerABTesting:
    def __init__(self):
        self.sagemaker = boto3.client('sagemaker')
        self.cloudwatch = boto3.client('cloudwatch')

    def create_multi_variant_endpoint(self, model_a_name, model_b_name):
        """Create endpoint with multiple model variants for A/B testing"""

        endpoint_config_name = f"ab-test-config-{datetime.now().strftime('%Y%m%d-%H%M%S')}"

        # Create endpoint configuration with two variants
        endpoint_config = self.sagemaker.create_endpoint_config(
            EndpointConfigName=endpoint_config_name,
            ProductionVariants=[
                {
                    'VariantName': 'variant-a',
                    'ModelName': model_a_name,
                    'InitialInstanceCount': 1,
                    'InstanceType': 'ml.m5.large',
                    'InitialVariantWeight': 70  # 70% traffic to model A
                },
                {
                    'VariantName': 'variant-b',
                    'ModelName': model_b_name,
                    'InitialInstanceCount': 1,
                    'InstanceType': 'ml.m5.large',
                    'InitialVariantWeight': 30  # 30% traffic to model B
                }
            ],
            DataCaptureConfig={
                'EnableCapture': True,
                'InitialSamplingPercentage': 100,
                'DestinationS3Uri': 's3://ml-data-capture/ab-test/',
                'CaptureOptions': [
                    {'CaptureMode': 'Input'},
                    {'CaptureMode': 'Output'}
                ]
            }
        )
```

```
        # Create endpoint
        endpoint_name = f"ab-test-endpoint-{datetime.now().strftime('%Y%m%d-%H%M%S')}"

        endpoint = self.sagemaker.create_endpoint(
            EndpointName=endpoint_name,
            EndpointConfigName=endpoint_config_name
        )

        return endpoint_name, endpoint_config_name

    def monitor_ab_test_metrics(self, endpoint_name, days=7):
        """Monitor A/B test metrics using CloudWatch"""

        end_time = datetime.utcnow()
        start_time = end_time - timedelta(days=days)

        metrics_to_monitor = [
            'Invocations',
            'Invocation4XXErrors',
            'Invocation5XXErrors',
            'ModelLatency'
        ]

        results = {}

        for variant in ['variant-a', 'variant-b']:
            variant_metrics = {}

            for metric_name in metrics_to_monitor:
                response = self.cloudwatch.get_metric_statistics(
                    Namespace='AWS/SageMaker',
                    MetricName=metric_name,
                    Dimensions=[
                        {'Name': 'EndpointName', 'Value': endpoint_name},
                        {'Name': 'VariantName', 'Value': variant}
                    ],
                    StartTime=start_time,
                    EndTime=end_time,
                    Period=3600,  # 1 hour periods
                    Statistics=['Sum', 'Average', 'Maximum']
                )

                variant_metrics[metric_name] = response['Datapoints']

            results[variant] = variant_metrics

        return results

    def analyze_ab_test_results(self, endpoint_name, business_metric_data):
        """Analyze A/B test results with statistical significance"""

        import scipy.stats as stats
        import numpy as np

        # Get technical metrics from CloudWatch
```

```
        technical_metrics = self.monitor_ab_test_metrics(endpoint_name)

        # Combine with business metrics
        analysis_results = {}

        for variant in ['variant-a', 'variant-b']:
            variant_data = business_metric_data[variant]

            analysis_results[variant] = {
                'sample_size': len(variant_data['conversion_rates']),
                'conversion_rate': np.mean(variant_data['conversion_rates']),
                'conversion_rate_std': np.std(variant_data['conversion_rates']),
                'average_latency': np.mean([
                    dp['Average'] for dp in technical_metrics[variant]['ModelLatency']
                ]),
                'total_invocations': sum([
                    dp['Sum'] for dp in technical_metrics[variant]['Invocations']
                ])
            }

        # Statistical significance test
        variant_a_conversions = business_metric_data['variant-a']['conversion_rates']
        variant_b_conversions = business_metric_data['variant-b']['conversion_rates']

        t_stat, p_value = stats.ttest_ind(variant_a_conversions, variant_b_conversions)

        analysis_results['statistical_test'] = {
            't_statistic': t_stat,
            'p_value': p_value,
            'is_significant': p_value &lt; 0.05,
            'confidence_level': 0.95
        }

        # Effect size calculation
        pooled_std = np.sqrt((
            np.var(variant_a_conversions) + np.var(variant_b_conversions)
        ) / 2)

        cohens_d = (
            np.mean(variant_b_conversions) - np.mean(variant_a_conversions)
        ) / pooled_std

        analysis_results['effect_size'] = {
            'cohens_d': cohens_d,
            'interpretation': (
                'large' if abs(cohens_d) &gt; 0.8 else
                'medium' if abs(cohens_d) &gt; 0.5 else
                'small'
            )
        }

        return analysis_results

    def update_traffic_distribution(self, endpoint_name, variant_a_weight, variant_b_weig
        """Update traffic distribution based on A/B test results"""
```

```
        # Get current endpoint configuration
        describe_response = self.sagemaker.describe_endpoint(
            EndpointName=endpoint_name
        )
        current_config = describe_response['EndpointConfigName']

        # Create new endpoint configuration with updated weights
        new_config_name = f"updated-config-{datetime.now().strftime('%Y%m%d-%H%M%S')}"

        # Get current configuration details
        config_details = self.sagemaker.describe_endpoint_config(
            EndpointConfigName=current_config
        )

        # Update variant weights
        updated_variants = config_details['ProductionVariants']
        updated_variants[0]['InitialVariantWeight'] = variant_a_weight
        updated_variants[1]['InitialVariantWeight'] = variant_b_weight

        # Create new configuration
        self.sagemaker.create_endpoint_config(
            EndpointConfigName=new_config_name,
            ProductionVariants=updated_variants
        )

        # Update endpoint
        self.sagemaker.update_endpoint(
            EndpointName=endpoint_name,
            EndpointConfigName=new_config_name
        )

        return new_config_name

# Usage example
def run_ab_test():
    ab_tester = SageMakerABTesting()

    # Deploy A/B test endpoint
    endpoint_name, config_name = ab_tester.create_multi_variant_endpoint(
        'fraud-model-v1', 'fraud-model-v2'
    )

    # Simulate business metrics collection (in practice, collect from your application)
    business_metrics = {
        'variant-a': {
            'conversion_rates': np.random.normal(0.15, 0.05, 1000)  # 15% baseline
        },
        'variant-b': {
            'conversion_rates': np.random.normal(0.18, 0.05, 1000)  # 18% new model
        }
    }

    # Analyze results after collecting sufficient data
    results = ab_tester.analyze_ab_test_results(endpoint_name, business_metrics)

    if results['statistical_test']['is_significant'] and results['effect_size']['cohens_d
```

```
        print("Model B performs significantly better. Shifting more traffic.")
        ab_tester.update_traffic_distribution(endpoint_name, 20, 80)  # 80% to variant B

    return results
```

Data Streaming & Processing (Kinesis, MSK)

Q13: How do you build a real-time data pipeline using Kinesis?

Answer:

Kinesis Real-time Pipeline Architecture:

1. Kinesis Data Streams Setup:

```
import boto3
import json
import time
from datetime import datetime
import base64

class KinesisDataPipeline:
    def __init__(self):
        self.kinesis = boto3.client('kinesis')
        self.firehose = boto3.client('firehose')
        self.analytics = boto3.client('kinesisanalytics')

    def create_data_stream(self, stream_name, shard_count=2):
        """Create Kinesis data stream for real-time data ingestion"""

        try:
            response = self.kinesis.create_stream(
                StreamName=stream_name,
                ShardCount=shard_count
            )

            # Wait for stream to be active
            waiter = self.kinesis.get_waiter('stream_exists')
            waiter.wait(StreamName=stream_name)

            print(f"Stream {stream_name} created successfully")
            return response

        except Exception as e:
            print(f"Error creating stream: {e}")
            return None

    def put_records_batch(self, stream_name, records):
        """Batch insert records to Kinesis stream"""

        kinesis_records = []
        for record in records:
            kinesis_records.append({
```

```
                'Data': json.dumps(record),
                'PartitionKey': record.get('user_id', 'default')
            })

        # Batch put records (up to 500 records per batch)
        batch_size = 500
        for i in range(0, len(kinesis_records), batch_size):
            batch = kinesis_records[i:i + batch_size]

            response = self.kinesis.put_records(
                Records=batch,
                StreamName=stream_name
            )

            # Handle failed records
            failed_records = [
                batch[i] for i, record in enumerate(response['Records'])
                if 'ErrorCode' in record
            ]

            if failed_records:
                print(f"Failed to insert {len(failed_records)} records")
                # Implement retry logic here

        return response

    def create_firehose_delivery_stream(self, delivery_stream_name, s3_bucket, s3_prefix)
        """Create Kinesis Data Firehose for S3 delivery"""

        firehose_config = {
            'DeliveryStreamName': delivery_stream_name,
            'DeliveryStreamType': 'DirectPut',
            'ExtendedS3DestinationConfiguration': {
                'RoleARN': 'arn:aws:iam::account:role/firehose_delivery_role',
                'BucketARN': f'arn:aws:s3:::{s3_bucket}',
                'Prefix': f'{s3_prefix}/year=!{timestamp:yyyy}/month=!{timestamp:MM}/day=
                'ErrorOutputPrefix': 'errors/',
                'BufferingHints': {
                    'SizeInMBs': 5,
                    'IntervalInSeconds': 300
                },
                'CompressionFormat': 'GZIP',
                'DataFormatConversionConfiguration': {
                    'Enabled': True,
                    'OutputFormatConfiguration': {
                        'Serializer': {
                            'ParquetSerDe': {}
                        }
                    },
                    'SchemaConfiguration': {
                        'DatabaseName': 'ml_database',
                        'TableName': 'streaming_data',
                        'RoleARN': 'arn:aws:iam::account:role/firehose_delivery_role'
                    }
                },
                'ProcessingConfiguration': {
```

```
                    'Enabled': True,
                    'Processors': [
                        {
                            'Type': 'Lambda',
                            'Parameters': [
                                {
                                    'ParameterName': 'LambdaArn',
                                    'ParameterValue': 'arn:aws:lambda:region:account:func
                                }
                            ]
                        }
                    ]
                }
            }
        }

        response = self.firehose.create_delivery_stream(**firehose_config)
        return response

    def create_lambda_processor(self):
        """Lambda function for real-time data transformation"""

        lambda_code = '''
import json
import base64
from datetime import datetime

def lambda_handler(event, context):
    output = []

    for record in event['records']:
        # Decode the data
        payload = base64.b64decode(record['data']).decode('utf-8')
        data = json.loads(payload)

        # Data transformation and enrichment
        transformed_data = {
            'user_id': data.get('user_id'),
            'transaction_id': data.get('transaction_id'),
            'amount': float(data.get('amount', 0)),
            'category': data.get('category', 'unknown'),
            'timestamp': data.get('timestamp', datetime.now().isoformat()),

            # Feature engineering
            'hour_of_day': datetime.fromisoformat(data.get('timestamp')).hour,
            'is_weekend': datetime.fromisoformat(data.get('timestamp')).weekday() &gt;= 5
            'amount_log': math.log1p(float(data.get('amount', 0))),

            # Data quality flags
            'is_valid': validate_transaction(data),
            'processing_timestamp': datetime.now().isoformat()
        }

        # Encode transformed data
        output_record = {
            'recordId': record['recordId'],
```

```
            'result': 'Ok',
            'data': base64.b64encode(
                json.dumps(transformed_data).encode('utf-8')
            ).decode('utf-8')
        }

        output.append(output_record)

    return {'records': output}

def validate_transaction(data):
    """Basic data validation"""
    required_fields = ['user_id', 'transaction_id', 'amount']

    for field in required_fields:
        if field not in data or data[field] is None:
            return False

    if float(data.get('amount', 0)) &lt;= 0:
        return False

    return True
        '''

        return lambda_code

# Real-time ML inference with Kinesis
class KinesisMLInference:
    def __init__(self, stream_name, sagemaker_endpoint):
        self.kinesis = boto3.client('kinesis')
        self.sagemaker = boto3.client('sagemaker-runtime')
        self.stream_name = stream_name
        self.endpoint_name = sagemaker_endpoint

    def process_stream_records(self):
        """Process streaming records and perform ML inference"""

        # Get shard iterator
        response = self.kinesis.describe_stream(StreamName=self.stream_name)
        shard_id = response['StreamDescription']['Shards'][0]['ShardId']

        shard_iterator_response = self.kinesis.get_shard_iterator(
            StreamName=self.stream_name,
            ShardId=shard_id,
            ShardIteratorType='LATEST'
        )

        shard_iterator = shard_iterator_response['ShardIterator']

        while True:
            # Get records from stream
            response = self.kinesis.get_records(
                ShardIterator=shard_iterator,
                Limit=100
            )
```

```
            records = response['Records']

            if records:
                # Process batch of records
                self.process_batch(records)

            # Update shard iterator
            shard_iterator = response['NextShardIterator']

            time.sleep(1)  # Avoid throttling

    def process_batch(self, records):
        """Process batch of records for ML inference"""

        batch_data = []
        record_metadata = []

        for record in records:
            data = json.loads(record['Data'])

            # Extract features for ML model
            features = [
                data.get('amount_log', 0),
                data.get('hour_of_day', 0),
                data.get('is_weekend', 0),
                data.get('category_encoded', 0)
            ]

            batch_data.append(features)
            record_metadata.append({
                'user_id': data.get('user_id'),
                'transaction_id': data.get('transaction_id'),
                'partition_key': record['PartitionKey'],
                'sequence_number': record['SequenceNumber']
            })

        if batch_data:
            # Perform batch inference
            predictions = self.batch_predict(batch_data)

            # Process predictions
            self.handle_predictions(predictions, record_metadata)

    def batch_predict(self, features_batch):
        """Perform batch ML inference using SageMaker endpoint"""

        # Convert to CSV format for SageMaker
        csv_data = '\n'.join([','.join(map(str, features)) for features in features_batch

        response = self.sagemaker.invoke_endpoint(
            EndpointName=self.endpoint_name,
            ContentType='text/csv',
            Body=csv_data
        )

        # Parse predictions
```

```
        result = response['Body'].read().decode('utf-8')
        predictions = [float(pred) for pred in result.strip().split('\n')]

        return predictions

    def handle_predictions(self, predictions, metadata):
        """Handle ML predictions (store to DynamoDB, send alerts, etc.)"""

        dynamodb = boto3.resource('dynamodb')
        table = dynamodb.Table('real-time-predictions')

        with table.batch_writer() as batch:
            for i, (prediction, meta) in enumerate(zip(predictions, metadata)):

                # High-risk transaction alert
                if prediction &gt; 0.8:  # High fraud probability
                    self.send_fraud_alert(meta, prediction)

                # Store prediction
                batch.put_item(
                    Item={
                        'transaction_id': meta['transaction_id'],
                        'user_id': meta['user_id'],
                        'fraud_score': prediction,
                        'timestamp': datetime.now().isoformat(),
                        'model_version': 'v1.0'
                    }
                )

    def send_fraud_alert(self, metadata, prediction):
        """Send real-time fraud alert"""
        sns = boto3.client('sns')

        message = {
            'alert_type': 'high_risk_transaction',
            'transaction_id': metadata['transaction_id'],
            'user_id': metadata['user_id'],
            'fraud_score': prediction,
            'timestamp': datetime.now().isoformat()
        }

        sns.publish(
            TopicArn='arn:aws:sns:region:account:fraud-alerts',
            Message=json.dumps(message),
            Subject='High Risk Transaction Detected'
        )

# Usage example
def deploy_kinesis_pipeline():
    pipeline = KinesisDataPipeline()

    # Create data stream
    pipeline.create_data_stream('transaction-stream', shard_count=4)

    # Create Firehose delivery stream
    pipeline.create_firehose_delivery_stream(
```

```
        'transaction-firehose',
        'data-lake-bucket',
        'streaming-data'
    )

    # Start real-time ML inference
    ml_processor = KinesisMLInference(
        'transaction-stream',
        'fraud-detection-endpoint'
    )

    # This would run in a separate process/container
    # ml_processor.process_stream_records()

    return pipeline
```

Security & IAM

Q14: How do you implement security best practices for data science workloads?

Answer:

AWS Security Framework for Data Science:

1. IAM Policies and Roles:

```
import json
import boto3

class DataScienceSecurityManager:
    def __init__(self):
        self.iam = boto3.client('iam')
        self.sts = boto3.client('sts')

    def create_data_scientist_role(self):
        """Create least-privilege role for data scientists"""

        trust_policy = {
            "Version": "2012-10-17",
            "Statement": [
                {
                    "Effect": "Allow",
                    "Principal": {
                        "Service": "sagemaker.amazonaws.com"
                    },
                    "Action": "sts:AssumeRole"
                },
                {
                    "Effect": "Allow",
                    "Principal": {
                        "AWS": "arn:aws:iam::account:root"
                    },
                    "Action": "sts:AssumeRole",
```

```
                "Condition": {
                    "StringEquals": {
                        "aws:RequestedRegion": ["us-east-1", "us-west-2"]
                    },
                    "DateGreaterThan": {
                        "aws:CurrentTime": "2024-01-01T00:00:00Z"
                    }
                }
            }
        ]
    }

    # Data scientist permission policy
    data_scientist_policy = {
        "Version": "2012-10-17",
        "Statement": [
            {
                "Sid": "SageMakerFullAccess",
                "Effect": "Allow",
                "Action": [
                    "sagemaker:CreateTrainingJob",
                    "sagemaker:CreateModel",
                    "sagemaker:CreateEndpointConfig",
                    "sagemaker:CreateEndpoint",
                    "sagemaker:DescribeTrainingJob",
                    "sagemaker:DescribeModel",
                    "sagemaker:DescribeEndpoint",
                    "sagemaker:InvokeEndpoint"
                ],
                "Resource": "*",
                "Condition": {
                    "StringEquals": {
                        "sagemaker:InstanceTypes": [
                            "ml.t3.medium",
                            "ml.m5.large",
                            "ml.m5.xlarge"
                        ]
                    }
                }
            },
            {
                "Sid": "S3DataAccess",
                "Effect": "Allow",
                "Action": [
                    "s3:GetObject",
                    "s3:PutObject",
                    "s3:DeleteObject",
                    "s3:ListBucket"
                ],
                "Resource": [
                    "arn:aws:s3:::ml-data-bucket/*",
                    "arn:aws:s3:::ml-model-artifacts/*"
                ]
            },
            {
                "Sid": "SecretsManagerAccess",
```

```
                    "Effect": "Allow",
                    "Action": [
                        "secretsmanager:GetSecretValue"
                    ],
                    "Resource": "arn:aws:secretsmanager:*:*:secret:ml-database-credential
                },
                {
                    "Sid": "KMSAccess",
                    "Effect": "Allow",
                    "Action": [
                        "kms:Encrypt",
                        "kms:Decrypt",
                        "kms:ReEncrypt*",
                        "kms:GenerateDataKey*",
                        "kms:DescribeKey"
                    ],
                    "Resource": "arn:aws:kms:*:*:key/12345678-1234-1234-1234-123456789012
                }
            ]
        }

        # Create role
        self.iam.create_role(
            RoleName='DataScientistRole',
            AssumeRolePolicyDocument=json.dumps(trust_policy),
            Description='Role for data scientists with restricted permissions'
        )

        # Attach inline policy
        self.iam.put_role_policy(
            RoleName='DataScientistRole',
            PolicyName='DataScientistPolicy',
            PolicyDocument=json.dumps(data_scientist_policy)
        )

        return 'DataScientistRole'

    def setup_vpc_security(self):
        """Configure VPC with proper security groups for ML workloads"""

        ec2 = boto3.client('ec2')

        # Create security group for SageMaker
        sg_response = ec2.create_security_group(
            GroupName='sagemaker-sg',
            Description='Security group for SageMaker notebook instances',
            VpcId='vpc-12345678'
        )

        security_group_id = sg_response['GroupId']

        # Inbound rules (restrictive)
        ec2.authorize_security_group_ingress(
            GroupId=security_group_id,
            IpPermissions=[
                {
```

```
                    'IpProtocol': 'tcp',
                    'FromPort': 443,
                    'ToPort': 443,
                    'IpRanges': [
                        {'CidrIp': '10.0.0.0/8', 'Description': 'Internal network HTTPS'}
                    ]
                }
            ]
        )

        # Outbound rules (specific endpoints)
        ec2.authorize_security_group_egress(
            GroupId=security_group_id,
            IpPermissions=[
                {
                    'IpProtocol': 'tcp',
                    'FromPort': 443,
                    'ToPort': 443,
                    'IpRanges': [
                        {'CidrIp': '0.0.0.0/0', 'Description': 'HTTPS for AWS services'}
                    ]
                }
            ]
        )

        return security_group_id

    def setup_data_encryption(self):
        """Configure encryption for data at rest and in transit"""

        kms = boto3.client('kms')

        # Create KMS key for ML data
        key_policy = {
            "Version": "2012-10-17",
            "Statement": [
                {
                    "Sid": "Enable IAM User Permissions",
                    "Effect": "Allow",
                    "Principal": {
                        "AWS": f"arn:aws:iam::{boto3.client('sts').get_caller_identity()[
                    },
                    "Action": "kms:*",
                    "Resource": "*"
                },
                {
                    "Sid": "Allow use of the key for ML services",
                    "Effect": "Allow",
                    "Principal": {
                        "Service": [
                            "sagemaker.amazonaws.com",
                            "s3.amazonaws.com"
                        ]
                    },
                    "Action": [
                        "kms:Encrypt",
```

```
                        "kms:Decrypt",
                        "kms:ReEncrypt*",
                        "kms:GenerateDataKey*",
                        "kms:DescribeKey"
                    ],
                    "Resource": "*"
                }
            ]
        }

        key_response = kms.create_key(
            Policy=json.dumps(key_policy),
            Description='KMS key for ML data encryption',
            Usage='ENCRYPT_DECRYPT'
        )

        key_id = key_response['KeyMetadata']['KeyId']

        # Create alias for the key
        kms.create_alias(
            AliasName='alias/ml-data-key',
            TargetKeyId=key_id
        )

        return key_id

    def setup_secrets_management(self):
        """Configure secrets management for database credentials"""

        secrets_manager = boto3.client('secretsmanager')

        secret_value = {
            "username": "ml_user",
            "password": "generated_password_here",
            "engine": "postgres",
            "host": "ml-database.cluster-xyz.us-east-1.rds.amazonaws.com",
            "port": 5432,
            "dbname": "ml_database"
        }

        secret_response = secrets_manager.create_secret(
            Name='ml-database-credentials',
            Description='Database credentials for ML workloads',
            SecretString=json.dumps(secret_value),
            KmsKeyId='alias/ml-data-key'
        )

        return secret_response['ARN']

# Data classification and protection
class DataGovernanceManager:
    def __init__(self):
        self.s3 = boto3.client('s3')

    def classify_and_tag_data(self, bucket_name):
        """Implement data classification and tagging"""
```

```
        # S3 bucket tagging for data classification
        tagging = {
            'TagSet': [
                {'Key': 'DataClassification', 'Value': 'Confidential'},
                {'Key': 'Environment', 'Value': 'Production'},
                {'Key': 'Owner', 'Value': 'DataScience'},
                {'Key': 'Retention', 'Value': '7Years'},
                {'Key': 'Compliance', 'Value': 'PCI-DSS'}
            ]
        }

        self.s3.put_bucket_tagging(
            Bucket=bucket_name,
            Tagging=tagging
        )

        # Apply object-level tags
        paginator = self.s3.get_paginator('list_objects_v2')

        for page in paginator.paginate(Bucket=bucket_name):
            if 'Contents' in page:
                for obj in page['Contents']:
                    # Determine classification based on file path/name
                    classification = self.determine_classification(obj['Key'])

                    self.s3.put_object_tagging(
                        Bucket=bucket_name,
                        Key=obj['Key'],
                        Tagging={
                            'TagSet': [
                                {'Key': 'DataType', 'Value': classification['data_type']}
                                {'Key': 'SensitivityLevel', 'Value': classification['sens
                                {'Key': 'Purpose', 'Value': classification['purpose']}
                            ]
                        }
                    )

    def determine_classification(self, object_key):
        """Classify data based on file path and name patterns"""

        if 'pii' in object_key.lower() or 'personal' in object_key.lower():
            return {
                'data_type': 'PersonalData',
                'sensitivity': 'High',
                'purpose': 'Analytics'
            }
        elif 'transaction' in object_key.lower():
            return {
                'data_type': 'TransactionData',
                'sensitivity': 'Medium',
                'purpose': 'ML_Training'
            }
        else:
            return {
                'data_type': 'General',
```

```
                'sensitivity': 'Low',
                'purpose': 'Analytics'
            }

# Usage example
def implement_ml_security():
    security_manager = DataScienceSecurityManager()
    governance_manager = DataGovernanceManager()

    # Setup security components
    role_name = security_manager.create_data_scientist_role()
    security_group = security_manager.setup_vpc_security()
    kms_key = security_manager.setup_data_encryption()
    secret_arn = security_manager.setup_secrets_management()

    # Implement data governance
    governance_manager.classify_and_tag_data('ml-data-bucket')

    return {
        'role': role_name,
        'security_group': security_group,
        'kms_key': kms_key,
        'secret_arn': secret_arn
    }
```

Monitoring & Logging (CloudWatch, CloudTrail)

Q15: How do you monitor ML models and data pipelines in production?

Answer:

Comprehensive Monitoring Strategy:

1. CloudWatch Metrics and Alarms:

```
import boto3
from datetime import datetime, timedelta

class MLMonitoringManager:
    def __init__(self):
        self.cloudwatch = boto3.client('cloudwatch')
        self.logs = boto3.client('logs')
        self.sns = boto3.client('sns')

    def setup_sagemaker_monitoring(self, endpoint_name):
        """Setup comprehensive SageMaker endpoint monitoring"""

        # Define metrics to monitor
        metrics_config = [
            {
                'MetricName': 'Invocations',
                'Threshold': 1000,
                'ComparisonOperator': 'GreaterThanThreshold',
```

```
                'AlarmDescription': 'High number of endpoint invocations'
            },
            {
                'MetricName': 'ModelLatency',
                'Threshold': 5000,  # 5 seconds
                'ComparisonOperator': 'GreaterThanThreshold',
                'AlarmDescription': 'High model latency detected'
            },
            {
                'MetricName': 'Invocation4XXErrors',
                'Threshold': 10,
                'ComparisonOperator': 'GreaterThanThreshold',
                'AlarmDescription': 'High number of 4XX errors'
            },
            {
                'MetricName': 'Invocation5XXErrors',
                'Threshold': 5,
                'ComparisonOperator': 'GreaterThanThreshold',
                'AlarmDescription': 'Server errors detected'
            }
        ]

        # Create alarms
        for metric_config in metrics_config:
            alarm_name = f"{endpoint_name}-{metric_config['MetricName']}-Alarm"

            self.cloudwatch.put_metric_alarm(
                AlarmName=alarm_name,
                ComparisonOperator=metric_config['ComparisonOperator'],
                EvaluationPeriods=2,
                MetricName=metric_config['MetricName'],
                Namespace='AWS/SageMaker',
                Period=300,
                Statistic='Sum',
                Threshold=metric_config['Threshold'],
                ActionsEnabled=True,
                AlarmActions=[
                    'arn:aws:sns:region:account:ml-alerts'
                ],
                AlarmDescription=metric_config['AlarmDescription'],
                Dimensions=[
                    {
                        'Name': 'EndpointName',
                        'Value': endpoint_name
                    }
                ]
            )

    def create_custom_ml_metrics(self, model_name, predictions_data):
        """Create custom metrics for model performance monitoring"""

        import numpy as np
        from sklearn.metrics import accuracy_score, precision_score, recall_score

        # Calculate model performance metrics
        y_true = predictions_data['actual']
```

```
        y_pred = predictions_data['predicted']
        y_prob = predictions_data.get('probability', None)

        accuracy = accuracy_score(y_true, y_pred)
        precision = precision_score(y_true, y_pred, average='weighted')
        recall = recall_score(y_true, y_pred, average='weighted')

        # Data drift detection
        data_drift_score = self.calculate_data_drift(
            predictions_data['features'],
            predictions_data['baseline_features']
        )

        # Prediction drift detection
        pred_drift_score = self.calculate_prediction_drift(
            y_prob, predictions_data['baseline_predictions']
        )

        # Send custom metrics to CloudWatch
        custom_metrics = [
            {'MetricName': 'ModelAccuracy', 'Value': accuracy},
            {'MetricName': 'ModelPrecision', 'Value': precision},
            {'MetricName': 'ModelRecall', 'Value': recall},
            {'MetricName': 'DataDriftScore', 'Value': data_drift_score},
            {'MetricName': 'PredictionDriftScore', 'Value': pred_drift_score}
        ]

        for metric in custom_metrics:
            self.cloudwatch.put_metric_data(
                Namespace='ML/ModelPerformance',
                MetricData=[
                    {
                        'MetricName': metric['MetricName'],
                        'Value': metric['Value'],
                        'Unit': 'None',
                        'Dimensions': [
                            {
                                'Name': 'ModelName',
                                'Value': model_name
                            }
                        ]
                    }
                ]
            )

    def calculate_data_drift(self, current_features, baseline_features):
        """Calculate data drift score using statistical measures"""

        from scipy import stats
        import numpy as np

        drift_scores = []

        for i in range(current_features.shape[1]):
            current_col = current_features[:, i]
            baseline_col = baseline_features[:, i]
```

```
            # Kolmogorov-Smirnov test
            ks_statistic, p_value = stats.ks_2samp(current_col, baseline_col)
            drift_scores.append(ks_statistic)

        # Overall drift score (average of feature drift scores)
        return np.mean(drift_scores)

    def calculate_prediction_drift(self, current_predictions, baseline_predictions):
        """Calculate prediction drift score"""

        from scipy import stats

        if current_predictions is None or baseline_predictions is None:
            return 0.0

        # Earth Mover's Distance (Wasserstein distance)
        drift_score = stats.wasserstein_distance(
            current_predictions, baseline_predictions
        )

        return drift_score

    def setup_pipeline_monitoring(self, pipeline_name):
        """Monitor ETL/ML pipeline health"""

        # Create log group for pipeline
        log_group_name = f'/aws/lambda/ml-pipeline-{pipeline_name}'

        try:
            self.logs.create_log_group(logGroupName=log_group_name)
        except:
            pass  # Log group might already exist

        # Create metric filters for log analysis
        metric_filters = [
            {
                'filterName': 'ErrorCount',
                'filterPattern': 'ERROR',
                'metricTransformation': {
                    'metricName': 'PipelineErrors',
                    'metricNamespace': 'ML/Pipeline',
                    'metricValue': '1'
                }
            },
            {
                'filterName': 'ProcessingTime',
                'filterPattern': '[timestamp, requestId, level="INFO", message="Processin
                'metricTransformation': {
                    'metricName': 'ProcessingDuration',
                    'metricNamespace': 'ML/Pipeline',
                    'metricValue': '$duration'
                }
            }
        ]
```

```
        for filter_config in metric_filters:
            self.logs.put_metric_filter(
                logGroupName=log_group_name,
                **filter_config
            )

    def create_ml_dashboard(self, dashboard_name, endpoint_name, pipeline_name):
        """Create CloudWatch dashboard for ML monitoring"""

        dashboard_body = {
            "widgets": [
                {
                    "type": "metric",
                    "properties": {
                        "metrics": [
                            ["AWS/SageMaker", "Invocations", "EndpointName", endpoint_nam
                            [".", "ModelLatency", ".", "."],
                            [".", "Invocation4XXErrors", ".", "."],
                            [".", "Invocation5XXErrors", ".", "."]
                        ],
                        "period": 300,
                        "stat": "Average",
                        "region": "us-east-1",
                        "title": "SageMaker Endpoint Metrics"
                    }
                },
                {
                    "type": "metric",
                    "properties": {
                        "metrics": [
                            ["ML/ModelPerformance", "ModelAccuracy", "ModelName", endpoin
                            [".", "ModelPrecision", ".", "."],
                            [".", "ModelRecall", ".", "."]
                        ],
                        "period": 3600,
                        "stat": "Average",
                        "region": "us-east-1",
                        "title": "Model Performance Metrics"
                    }
                },
                {
                    "type": "metric",
                    "properties": {
                        "metrics": [
                            ["ML/ModelPerformance", "DataDriftScore", "ModelName", endpoi
                            [".", "PredictionDriftScore", ".", "."]
                        ],
                        "period": 3600,
                        "stat": "Average",
                        "region": "us-east-1",
                        "title": "Model Drift Detection"
                    }
                },
                {
                    "type": "log",
                    "properties": {
```

```
                        "query": f"SOURCE '/aws/lambda/ml-pipeline-{pipeline_name}'\n| fi
                        "region": "us-east-1",
                        "title": "Recent Pipeline Errors"
                    }
                }
            ]
        }

        self.cloudwatch.put_dashboard(
            DashboardName=dashboard_name,
            DashboardBody=json.dumps(dashboard_body)
        )

    def setup_automated_retraining_trigger(self, model_name, drift_threshold=0.1):
        """Setup automated retraining based on model drift"""

        # Create CloudWatch alarm for data drift
        self.cloudwatch.put_metric_alarm(
            AlarmName=f'{model_name}-DataDrift-Alarm',
            ComparisonOperator='GreaterThanThreshold',
            EvaluationPeriods=2,
            MetricName='DataDriftScore',
            Namespace='ML/ModelPerformance',
            Period=3600,
            Statistic='Average',
            Threshold=drift_threshold,
            ActionsEnabled=True,
            AlarmActions=[
                'arn:aws:sns:region:account:ml-retraining-trigger'
            ],
            AlarmDescription=f'Trigger retraining for {model_name} due to data drift',
            Dimensions=[
                {
                    'Name': 'ModelName',
                    'Value': model_name
                }
            ]
        )

# Model explainability monitoring
class ModelExplainabilityMonitor:
    def __init__(self):
        self.s3 = boto3.client('s3')

    def setup_sagemaker_clarify_monitoring(self, endpoint_name, baseline_job_name):
        """Setup SageMaker Clarify for bias and explainability monitoring"""

        sagemaker = boto3.client('sagemaker')

        monitoring_schedule_config = {
            'MonitoringScheduleName': f'{endpoint_name}-bias-monitoring',
            'MonitoringScheduleConfig': {
                'ScheduleConfig': {
                    'ScheduleExpression': 'cron(0 */6 * * ? *)'  # Every 6 hours
                },
                'MonitoringJobDefinition': {
```

```
                    'BaselineConfig': {
                        'BaseliningJobName': baseline_job_name
                    },
                    'MonitoringAppSpecification': {
                        'ImageUri': '205585389593.dkr.ecr.us-east-1.amazonaws.com/sagemak
                    },
                    'MonitoringInputs': [
                        {
                            'EndpointInput': {
                                'EndpointName': endpoint_name,
                                'LocalPath': '/opt/ml/processing/input',
                                'S3DataDistributionType': 'FullyReplicated'
                            }
                        }
                    ],
                    'MonitoringOutputConfig': {
                        'MonitoringOutputs': [
                            {
                                'S3Output': {
                                    'S3Uri': f's3://ml-monitoring-bucket/bias-monitoring/
                                    'LocalPath': '/opt/ml/processing/output'
                                }
                            }
                        ]
                    },
                    'MonitoringResources': {
                        'ClusterConfig': {
                            'InstanceType': 'ml.m5.xlarge',
                            'InstanceCount': 1,
                            'VolumeSizeInGB': 20
                        }
                    },
                    'RoleArn': 'arn:aws:iam::account:role/SageMakerExecutionRole'
                }
            }
        }

        response = sagemaker.create_monitoring_schedule(**monitoring_schedule_config)
        return response

# Usage example
def setup_comprehensive_ml_monitoring():
    monitor = MLMonitoringManager()
    explainability_monitor = ModelExplainabilityMonitor()

    endpoint_name = 'fraud-detection-endpoint'
    pipeline_name = 'fraud-detection-pipeline'

    # Setup monitoring components
    monitor.setup_sagemaker_monitoring(endpoint_name)
    monitor.setup_pipeline_monitoring(pipeline_name)
    monitor.create_ml_dashboard('ML-Fraud-Detection-Dashboard', endpoint_name, pipeline_n
    monitor.setup_automated_retraining_trigger('fraud-detection-model')

    # Setup bias and explainability monitoring
    explainability_monitor.setup_sagemaker_clarify_monitoring(
```

```
        endpoint_name, 'fraud-detection-baseline-job'
    )

    return {
        'monitoring_setup': 'complete',
        'endpoint': endpoint_name,
        'dashboard': 'ML-Fraud-Detection-Dashboard'
    }
```

Quick Reference Guide

AWS Service Selection Matrix

```
service_selection = {
    'data_ingestion': {
        'batch': ['S3', 'DataSync', 'Snowball'],
        'streaming': ['Kinesis Data Streams', 'MSK'],
        'databases': ['DMS', 'Glue']
    },
    'data_processing': {
        'serverless': ['Lambda', 'Glue', 'Athena'],
        'managed': ['EMR', 'SageMaker Processing'],
        'containers': ['ECS', 'EKS', 'Batch']
    },
    'ml_development': {
        'managed': ['SageMaker', 'Rekognition', 'Comprehend'],
        'custom': ['EC2', 'ECS with GPU', 'Lambda']
    },
    'data_storage': {
        'data_lake': ['S3', 'Lake Formation'],
        'data_warehouse': ['Redshift', 'Redshift Spectrum'],
        'feature_store': ['SageMaker Feature Store', 'DynamoDB']
    }
}
```

Cost Optimization Checklist

1. **Use Spot Instances** for training jobs (up to 90% savings)
2. **Right-size instances** based on workload requirements
3. **S3 Lifecycle policies** for data archival
4. **Reserved instances** for predictable workloads
5. **Auto-scaling** for inference endpoints
6. **Data compression** and **Parquet format** for storage
7. **Batch inference** instead of real-time when possible

Security Best Practices

1. **Least privilege access** with IAM roles and policies
2. **VPC and security groups** for network isolation
3. **Encryption at rest and in transit** using KMS
4. **Secrets Manager** for credentials
5. **CloudTrail** for audit logging
6. **Data classification** and tagging
7. **Regular security reviews** and compliance checks

Performance Optimization Tips

1. **Use appropriate instance types** for workloads
2. **Enable data capture** for model monitoring
3. **Implement caching** strategies
4. **Optimize data formats** (Parquet vs CSV)
5. **Use content delivery networks** (CloudFront)
6. **Monitor and tune** regularly
7. **Implement circuit breakers** for resilience

This AWS Cloud Services cheat sheet provides comprehensive coverage of essential services for data science projects. Focus on understanding the integration patterns and when to use each service based on your specific requirements.